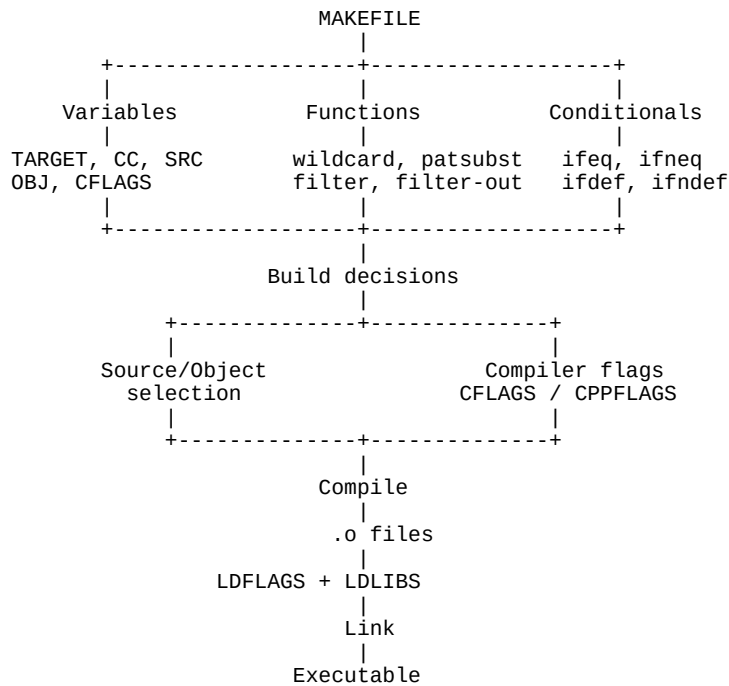


Makefile Concepts — Detailed Embedded Linux Reference

Practical study notes based on the concepts covered in the SampleKlcTest project. The goal is to understand existing Makefiles, .config-style configuration, and make small changes safely during Embedded Linux development and board bring-up.

1. Complete Concept Map



2. Makefile Core Structure

```
TARGET = SampleKlcTest
CC = gcc
OBJ = main.o add.o sub.o

all: $(TARGET)

$(TARGET): $(OBJ)
<TAB>$(CC) $^ -o $@

%.o: %.c
<TAB>$(CC) $(CFLAGS) -c $< -o $@
```

Target is what Make should build. **Prerequisites** are inputs/dependencies. **Recipe** is the command executed to build the target. Recipe lines normally begin with a TAB.

3. Variables

```
TARGET    = SampleKlcTest
CC        = gcc
SRC       = main.c add.c sub.c
OBJ       = main.o add.o sub.o
CFLAGS    = -Wall -O2 -g

$(TARGET) -> SampleKlcTest
$(CC)     -> gcc
$(SRC)    -> main.c add.c sub.c
$(OBJ)    -> main.o add.o sub.o
```

Variables keep the Makefile maintainable. If the compiler or target changes, you can change the variable instead of editing every recipe.

4. Automatic Variables

Variable	Meaning	Example
<code>\$@</code>	Current target	For main.o: main.c → main.o
<code>\$<</code>	First prerequisite	For main.o: main.c → main.c
<code>\$^</code>	All prerequisites	For target: a.o b.o c.o → a.o b.o c.o

```
$(TARGET): $(OBJ)
<TAB>$(CC) $^ -o $@
```

```
For this rule:
$@ = SampleKlcTest
$^ = main.o add.o sub.o
$< = main.o
```

5. Pattern Rules — `%.o: %.c`

```
%.o: %.c
<TAB>$(CC) $(CFLAGS) -c $< -o $@
```

The `%` is a pattern placeholder. Make can use one rule for many source/object pairs.

```
main.c -> main.o
add.c -> add.o
sub.c -> sub.o
mul.c -> mul.o
```

```
For main.o:
% = main
$< = main.c
$@ = main.o
```

```
Actual command:
gcc $(CFLAGS) -c main.c -o main.o
```

6. Make Functions

```
SRC = $(wildcard *.c)
OBJ = $(patsubst %.c,%.o,$(SRC))

wildcard:
    Finds matching files.
    *.c -> main.c add.c sub.c

patsubst:
    Transforms names.
    %.c -> %.o
    main.c add.c sub.c
    ->
    main.o add.o sub.o

filter:
    Selects matching words.

filter-out:
    Removes matching words.
    $(filter-out add.c,$(SRC))
```

7. Assignment Operators

Operator	Meaning	Typical use
=	Recursive assignment	SRC = \$(wildcard *.c)
:=	Immediate assignment	OBJ := \$(patsubst %.c,%.o,\$(SRC))
+=	Append	CFLAGS += -g
?=	Default if not already defined	DEBUG ?= yes

Example:

```
GCC_FLAGS = -c
GCC_FLAGS += -g
```

```
Result:
GCC_FLAGS = -c -g
```

```
Default:
DEBUG ?= yes
```

```
Command line:
make DEBUG=no
```

The command-line value can override the default.

8. Conditionals

```
ifeq ($(DEBUG),yes)
    CFLAGS += -g
endif

ifneq ($(DEBUG),yes)
    CFLAGS += -O2
endif

ifdef DEBUG
    # DEBUG is defined
endif

ifndef DEBUG
    # DEBUG is not defined
endif
```

ifeq/ifneq compare values. **ifdef/ifndef** test whether a variable is defined. For explicit y/n feature selection, **ifeq** is generally the easiest to read.

9. Feature Enable / Disable Directly in Makefile

```
DEBUG      ?= yes
ENABLE_ADD ?= yes
ENABLE_SUB ?= no

ifeq ($(DEBUG),yes)
```

```

CFLAGS += -g
endif

SRC = $(wildcard *.c)

ifeq ($(ENABLE_ADD),no)
SRC := $(filter-out add.c,$(SRC))
endif

ifeq ($(ENABLE_SUB),no)
SRC := $(filter-out sub.c,$(SRC))
endif

OBJ := $(patsubst %.c,%.o,$(SRC))

```

This is the first configuration model: one Makefile contains both configuration values and build logic.

10. Moving Configuration into .config

```

.config

CONFIG_DEBUG=y
CONFIG_ADD=y
CONFIG_SUB=n

Makefile:

include .config

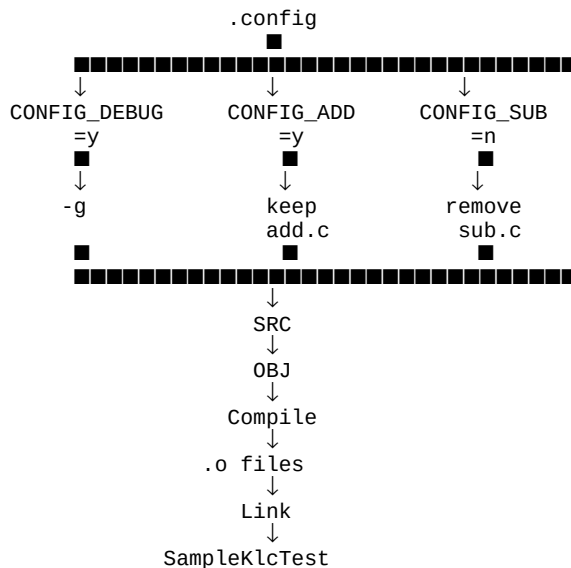
ifeq ($(CONFIG_DEBUG),y)
CFLAGS += -g
endif

ifeq ($(CONFIG_ADD),n)
SRC := $(filter-out add.c,$(SRC))
endif

ifeq ($(CONFIG_SUB),n)
SRC := $(filter-out sub.c,$(SRC))
endif

```

11. .config → Makefile → Build: Detailed Flow



Meaning: CONFIG_DEBUG=y adds debug information; CONFIG_ADD=y keeps add.c; CONFIG_SUB=n removes sub.c from the selected source list. The remaining source list becomes SRC, patsubst converts it to OBJ, the pattern rule compiles the objects, and the linker creates the executable.

12. Full SampleKlcTest Configuration-Driven Makefile

```
TARGET = SampleKlcTest
CC = gcc

include .config

SRC = $(wildcard *.c)

CFLAGS =

ifeq ($(CONFIG_DEBUG),y)
CFLAGS += -g
endif

ifeq ($(CONFIG_ADD),n)
SRC := $(filter-out add.c,$(SRC))
endif

ifeq ($(CONFIG_SUB),n)
SRC := $(filter-out sub.c,$(SRC))
endif

OBJ := $(patsubst %.c,%.o,$(SRC))

.PHONY: all clean print install uninstall

all: $(TARGET)

$(TARGET): $(OBJ)
<TAB>$(CC) $(LDFLAGS) $^ $(LDLIBS) -o $$@

%.o: %.c
<TAB>$(CC) $(CPPFLAGS) $(CFLAGS) -c $< -o $$@

print:
<TAB>@echo "SRC = $(SRC)"
<TAB>@echo "OBJ = $(OBJ)"
<TAB>@echo "CFLAGS = $(CFLAGS)"
<TAB>@echo "CONFIG_DEBUG = $(CONFIG_DEBUG)"
<TAB>@echo "CONFIG_ADD = $(CONFIG_ADD)"
<TAB>@echo "CONFIG_SUB = $(CONFIG_SUB)"

clean:
<TAB>rm -f *.o $(TARGET)

install:
<TAB>cp $(TARGET) /bin/

uninstall:
<TAB>rm -f /bin/$(TARGET)
```

13. Example Configuration Results

```
.config
CONFIG_DEBUG=y
CONFIG_ADD=y
CONFIG_SUB=n

Result:
SRC   = main.c add.c
OBJ   = main.o add.o
CFLAGS = -g

If changed to:

CONFIG_DEBUG=n
CONFIG_ADD=n
CONFIG_SUB=y

Result:
SRC   = main.c sub.c
OBJ   = main.o sub.o
CFLAGS =
```

14. CFLAGS / CPPFLAGS / LDFLAGS / LDLIBS

Variable	Stage / purpose	Examples
CFLAGS	C compiler options	-Wall -O2 -g

CPPFLAGS	Preprocessor options	-linclude -DDEBUG
LDLFLAGS	Linker options/search paths	-L./lib
LDLIBS	Libraries to link	-lm -lpthread

```
Compile:
gcc $(CPPFLAGS) $(CFLAGS) -c main.c -o main.o
```

```
Link:
gcc $(LDLFLAGS) main.o add.o $(LDLIBS) -o SampleKlcTest
```

15. .PHONY

`.PHONY: all clean install uninstall print`

Use `.PHONY` for action targets such as `clean`, `install`, `uninstall`, and `print`. It tells Make these names represent actions rather than files.

16. Useful make Commands

Command	What to remember
<code>make</code>	Build the default target.
<code>make print</code>	Print current Make variables.
<code>make clean</code>	Remove generated files according to the Makefile.
<code>make -n</code>	Show commands without executing them.
<code>make -B</code>	Force rebuilding.
<code>make -j4</code>	Build up to four jobs in parallel.
<code>make VAR=value</code>	Override a variable for this invocation.

17. Important Make Behavior: Timestamps

Make normally checks dependencies and timestamps.

```
If:
    main.c is older than main.o
```

```
then:
    main.o may be considered up-to-date.
```

Changing a Make variable such as `DEBUG=no` does not automatically force every object to rebuild.

```
Useful commands:
    make -B
    make clean && make
```

This matters when changing configuration or compiler flags. Always understand whether existing objects need to be rebuilt.

18. What You Need for Embedded Linux

You should be able to READ:

```
CC = arm-linux-gnueabi-gcc
CFLAGS += -O2 -g
CPPFLAGS += -I$(SDK)/include
LDFLAGS += -L$(SDK)/lib
LDLIBS += -lpthread

include .config

ifeq ($(CONFIG_DEBUG),y)
    CFLAGS += -g
endif

%.o: %.c
    $(CC) $(CPPFLAGS) $(CFLAGS) -c $< -o $@
```

You should be able to TRACE:

```
.config
↓
Make variables
↓
conditionals
↓
source/object selection
↓
compiler
↓
linker
↓
executable
```

19. What You Do NOT Need to Master Now

For your Embedded Linux learning path, you do not need to spend much time on advanced GNU Make metaprogramming, complex recursive Make systems, or writing a complete configuration framework. The practical target is to understand an existing build system and make minor changes confidently.

20. Makefile → .config → Kconfig/Kbuild Concept

Your learning model:

```
.config
↓
include .config
↓
ifeq(CONFIG_XXX, y/n)
↓
select flags / sources
↓
compile + link
```

Linux kernel concept:

```
Kconfig descriptions
↓
configuration interface
↓
.config
↓
Kbuild / Make
↓
kernel / modules / selected components
```

The first diagram is the simplified system you built for learning. The second is the broader Linux kernel configuration/build architecture. The concepts overlap, but the Linux kernel's Kconfig/Kbuild system is much more sophisticated.

21. Final Quick-Reference Sheet

AUTOMATIC VARIABLES
\$@ = current target
\$< = first prerequisite
\$^ = all prerequisites

FUNCTIONS
wildcard = find files

patsubst = transform names
filter = select
filter-out = remove

CONDITIONALS

ifeq / ifneq = compare values
ifdef / ifndef = check defined/not defined

ASSIGNMENT

= = recursive assignment
:= = immediate assignment
+= = append
?= = default if undefined

FLAGS

CFLAGS = C compiler options
CPPFLAGS = preprocessor options
LDFLAGS = linker options
LDLIBS = libraries

CONFIG

include .config
CONFIG_FEATURE=y
CONFIG_FEATURE=n

BUILD

.c -> .o -> executable

IMPORTANT COMMANDS

make
make print
make clean
make -n
make -B
make -j4
make VAR=value